

Designing High-Quality Contexts

A 7-Step Lifecycle Guide to
Operational AI Reliability

The Paradigm Shift: Prompting vs. Engineering

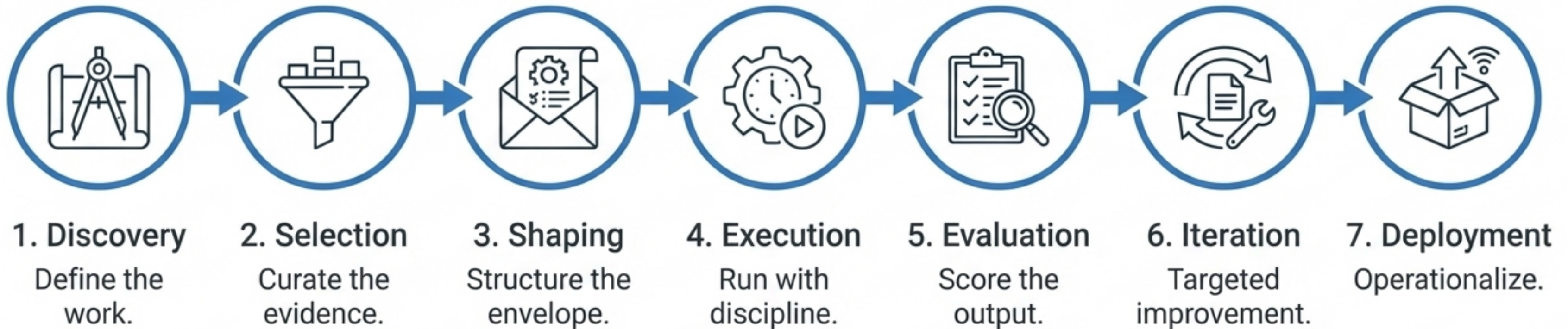
One-Shot Prompting

- ✗ Prompt writing is the starting point.
- ✗ Bloated context; pasting everything just in case.
- ✗ Random, unmeasured edits to fix weak outputs.
- ✗ Produces fragile, unrepeatable demos.

Context Engineering Lifecycle

- ✓ Prompt writing is the visible middle of a bounded workflow.
- ✓ The smallest sufficient evidence set.
- ✓ Phase-aware diagnosis and targeted iteration.
- ✓ Produces inspectable, operationally reliable assets.

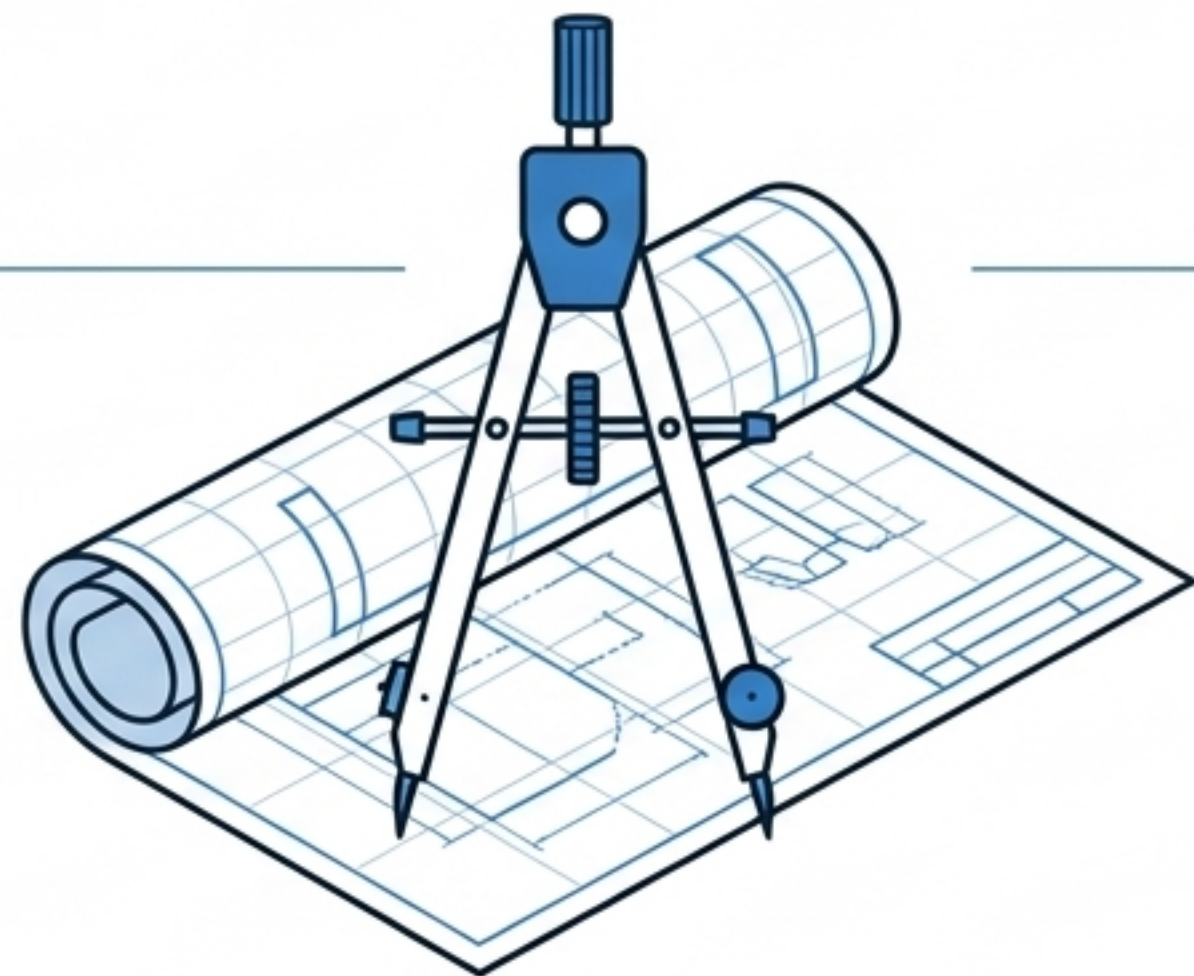
The Context Engineering Lifecycle



Phase 1: Discovery

Define the Job to be Done before writing to avoid hidden task inflation.

- Establish the problem statement, intended audience, and explicit constraints.
- Define an initial evaluation rubric.
- Develop representative test cases to prevent evaluating the package only on ideal, easy-to-solve scenarios.



Normal



Edge

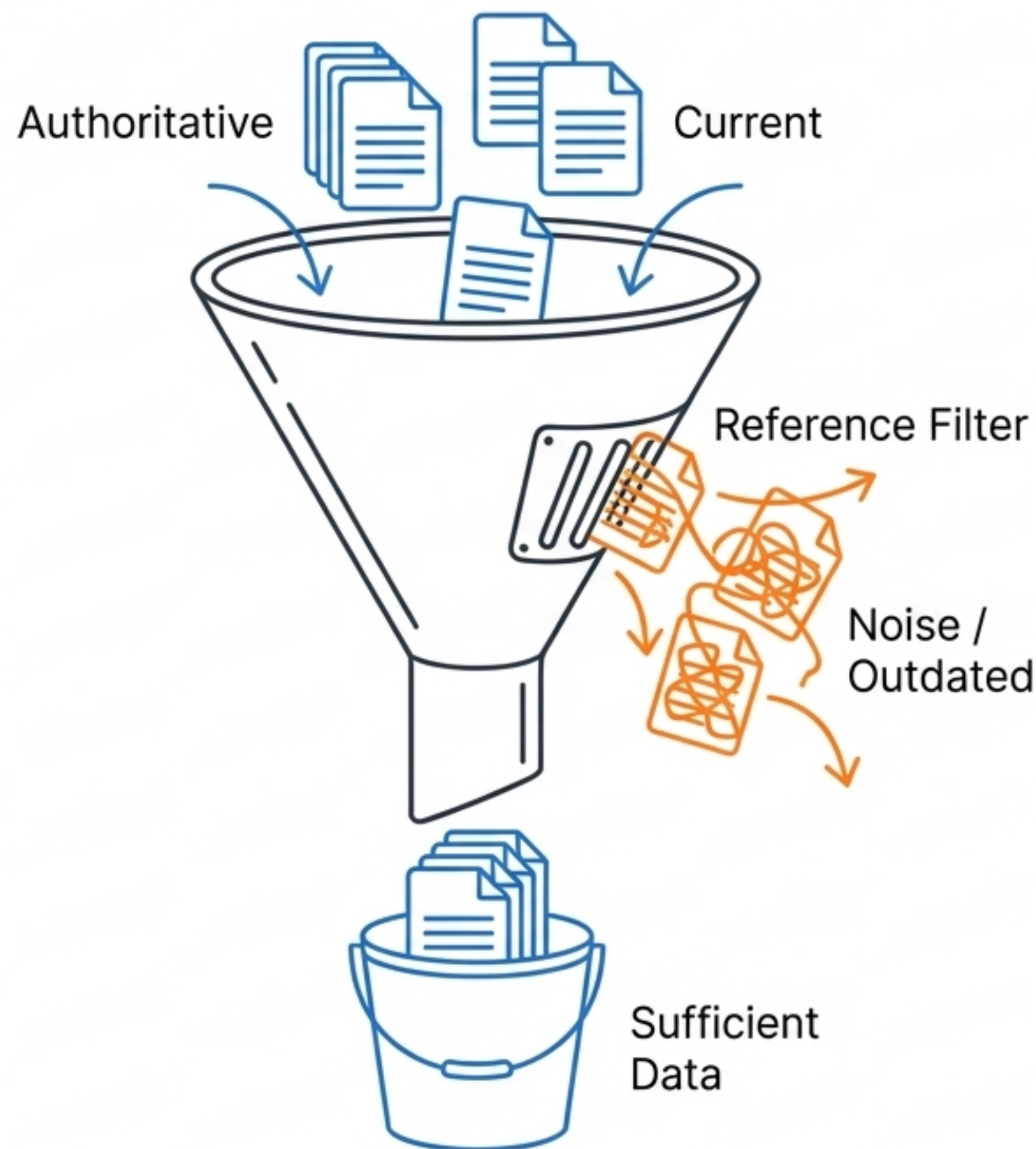


Failure-Prone

Phase 2: Selection

The Smallest Sufficient Evidence Set.

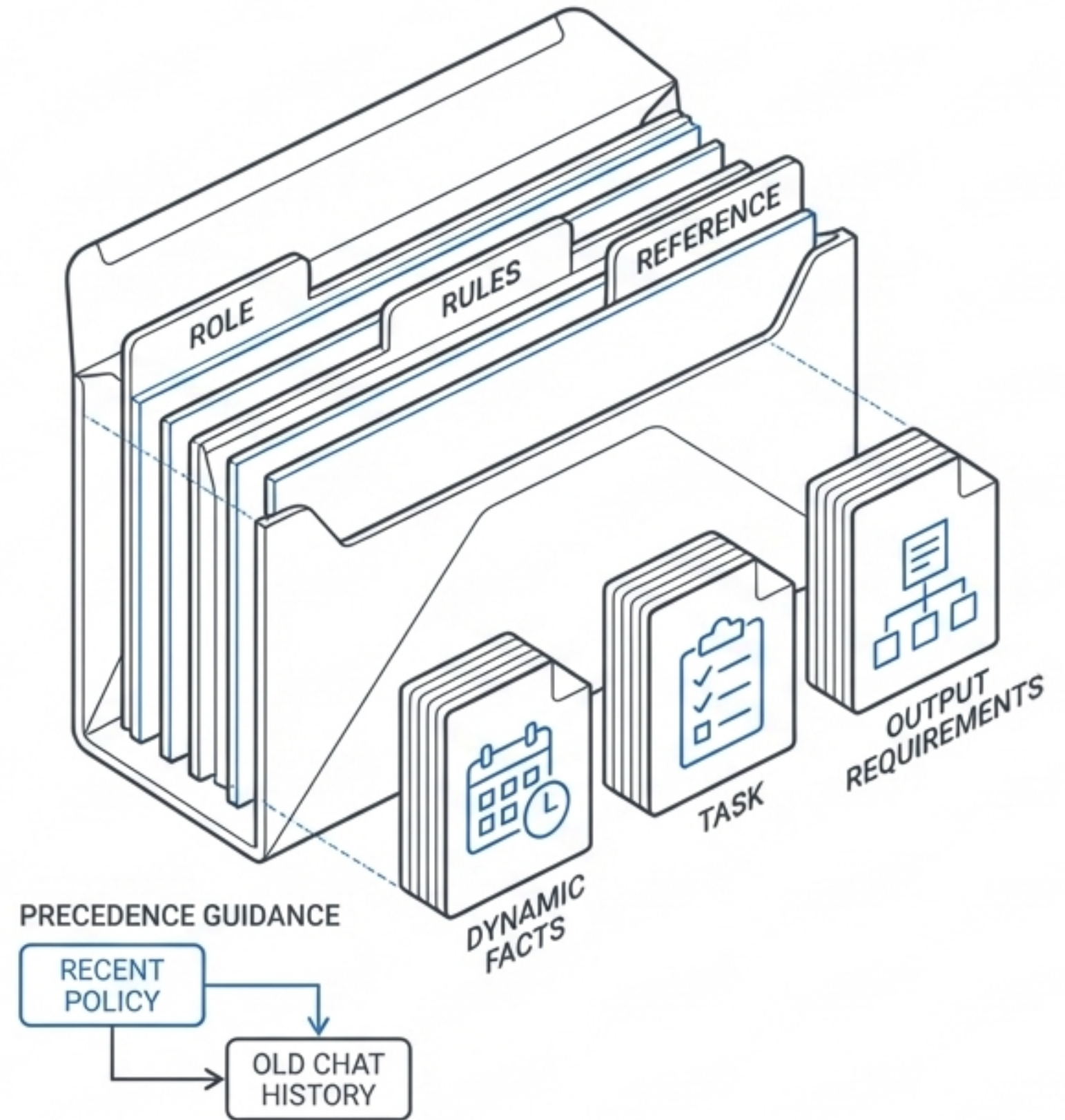
- Apply Token Budget discipline: include only the minimum set of authoritative material required for a correct answer.
- Apply the Relevance Filter: If removed, would the model guess? If no, remove it. Unnecessary text lowers the signal-to-noise ratio and creates distraction.



Phase 3: Shaping

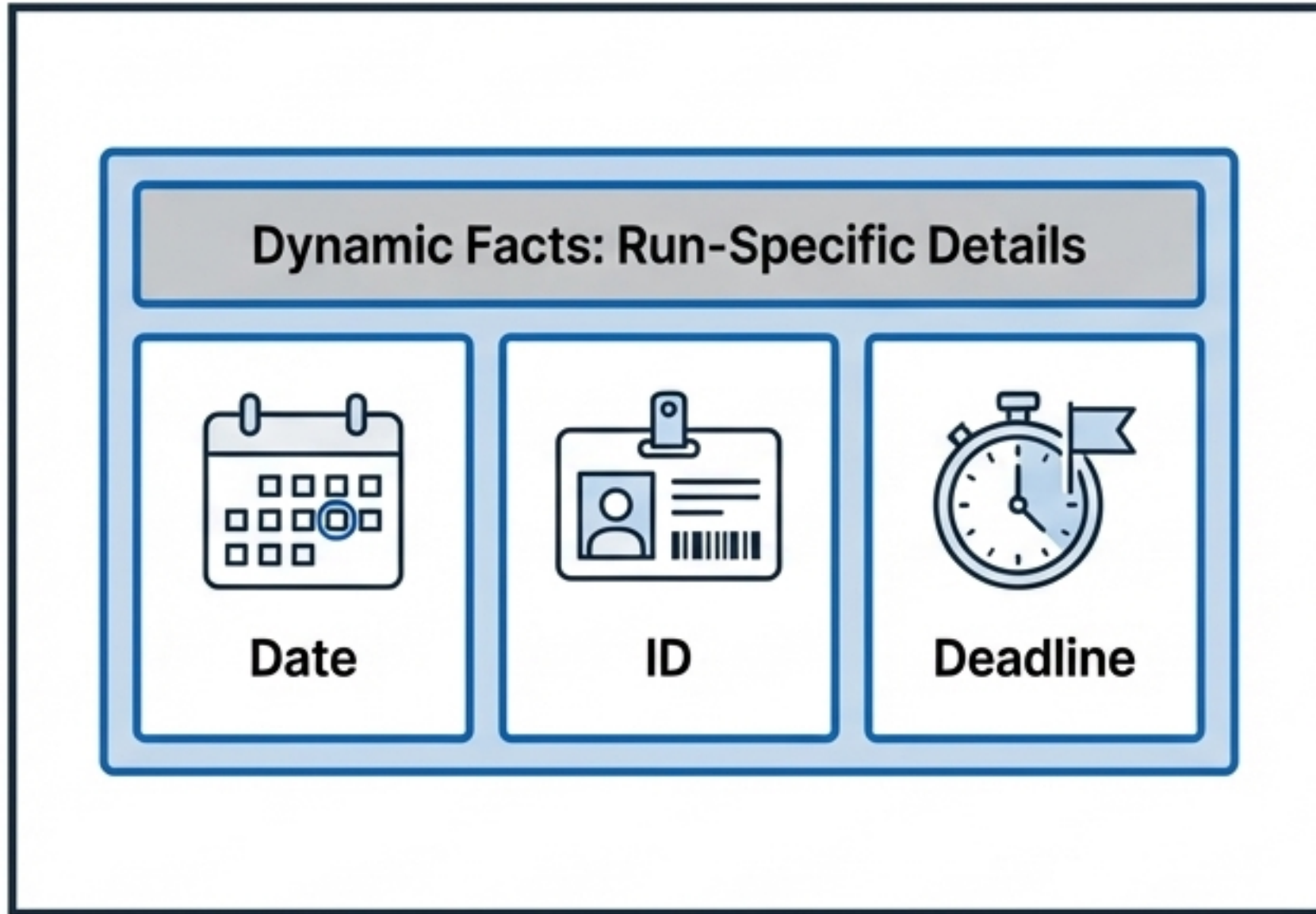
Structure the Context Envelope.

- Organize material into a legible operating brief by separating unlike things.
- Establish Precedence Guidance: Explicitly tell the model what wins in a conflict.
- Hit the Goldilocks Zone of instruction—specific enough to guide, but not rigidly over-engineered.

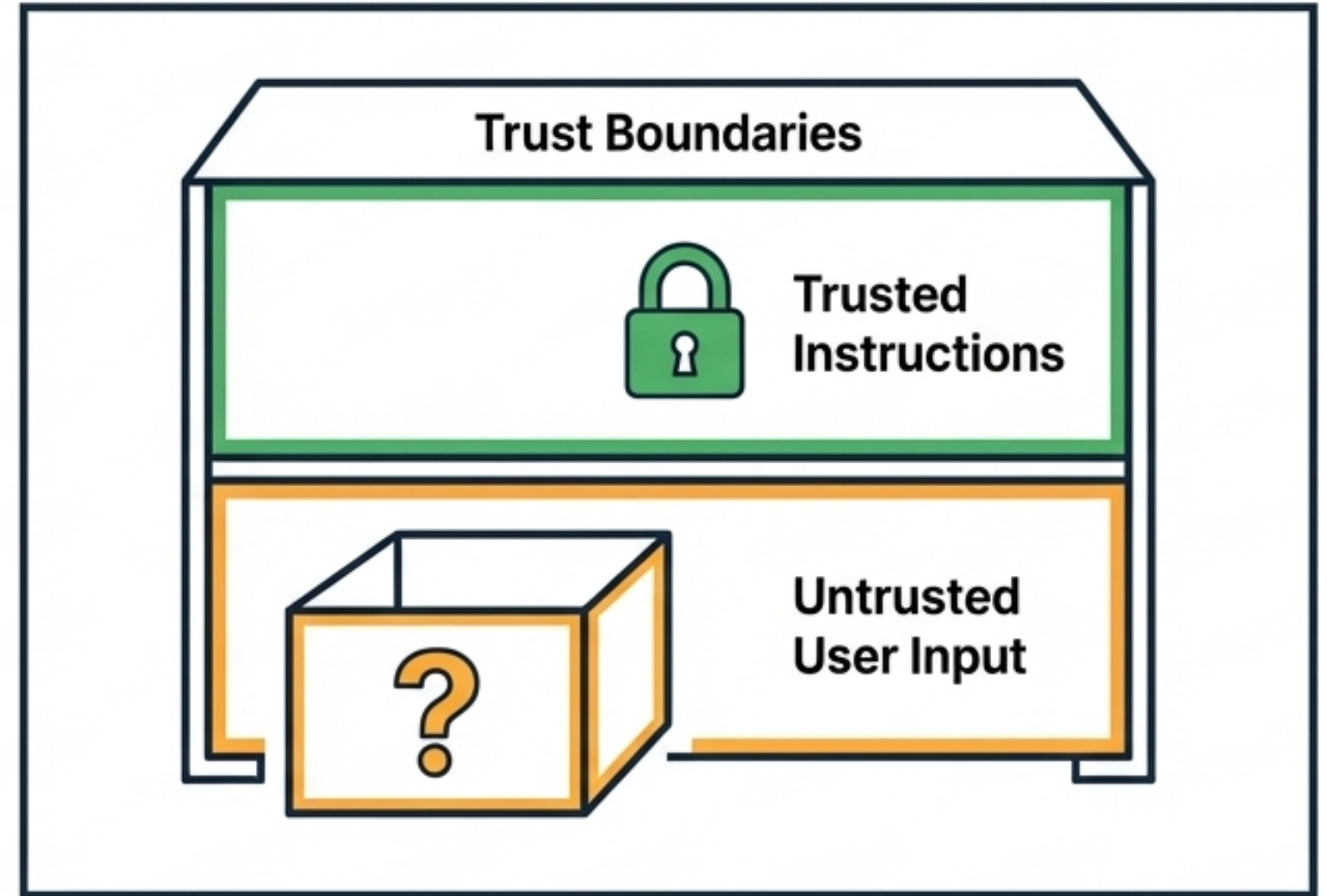


Phase 4: Execution

Current-State Control & Trust Boundaries.



Isolate Dynamic Facts: Place run-specific details in a dedicated block that is easily refreshed between runs.

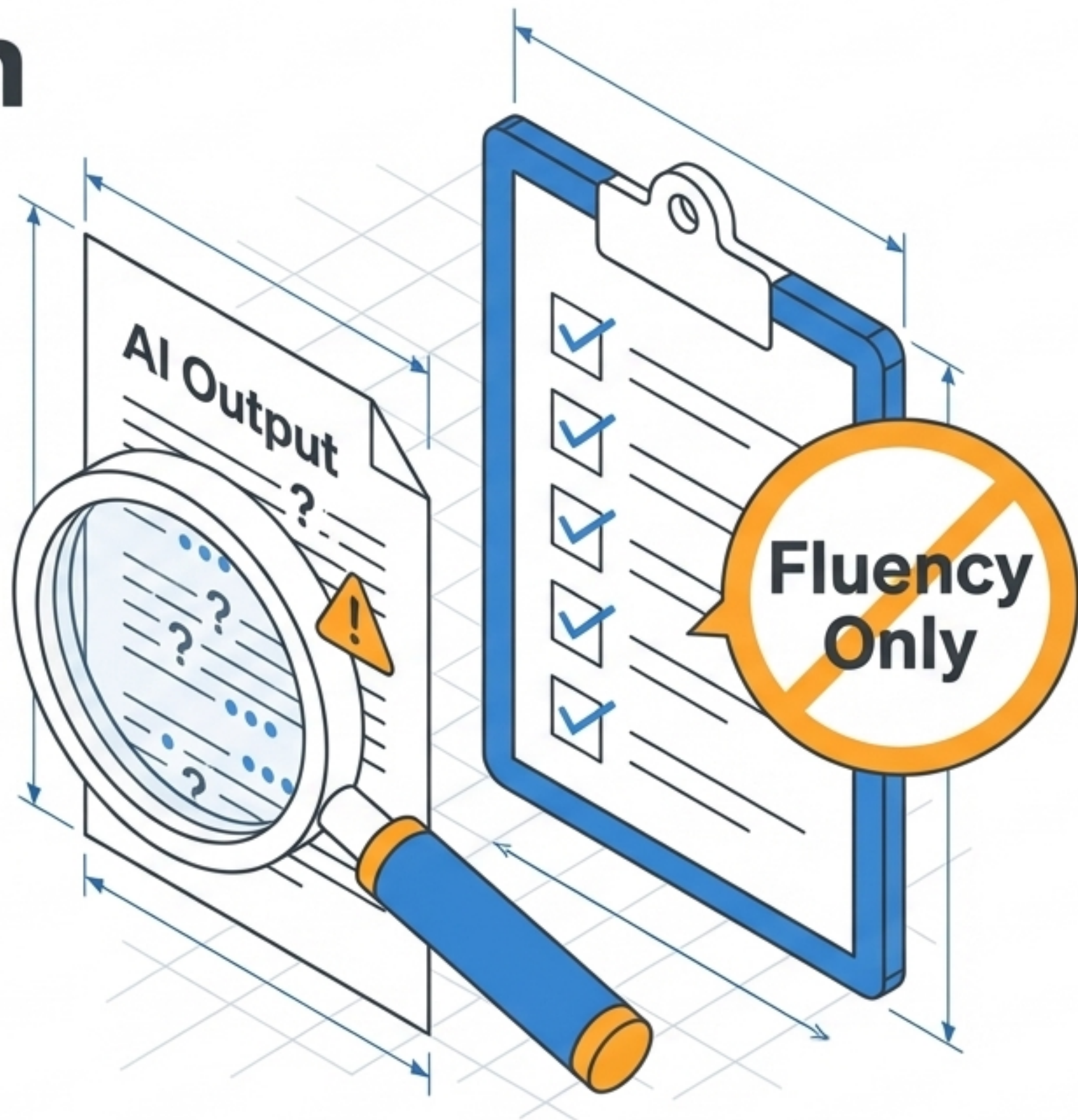


Enforce Trust Boundaries: Maintain strict separation between trusted rules and untrusted user-provided text to prevent prompt hijacking.

Phase 5: Evaluation

Groundedness & Constraint Adherence.

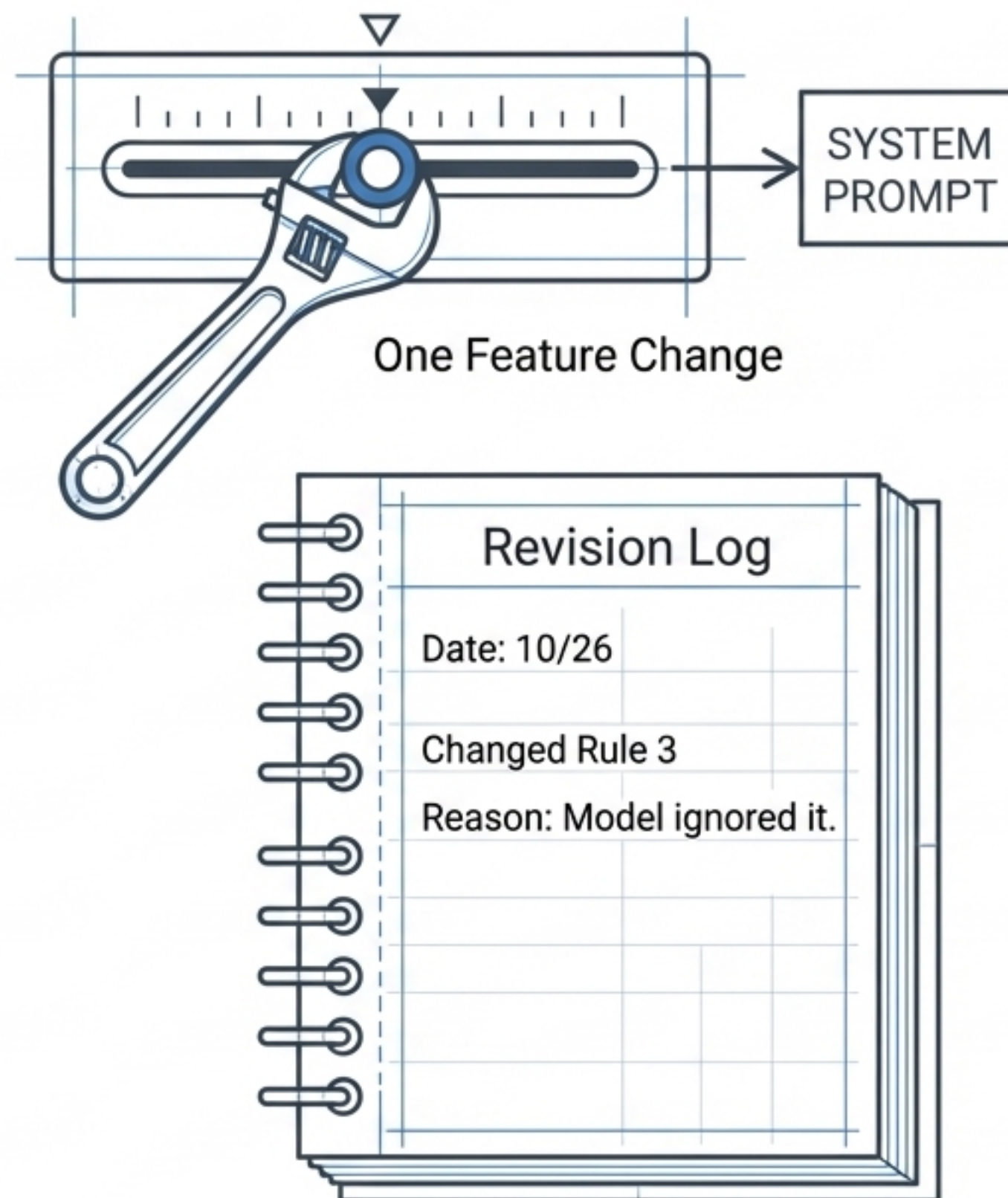
- Score outputs against the explicit rubric, not subjective impressions of fluency.
- Verify that the answer actually cited correct excerpts.
- Check that the model respected all safety and format boundaries rather than just sounding persuasive.



Phase 6: Iteration

Phase-Aware Diagnosis & Targeted Improvement.

- Fix the system, not just the single answer. If a rule was ignored, update the Shaping or Rules section rather than rewriting the whole prompt.
- Change one feature at a time to ensure interpretable results.
- Maintain a Revision Log to turn iteration from memory-based guesswork into an evidence-based discipline.



Phase 7: Deployment

Transition to a Managed Asset.

- Move from demo-ready to work-ready. A package is only deployed when it runs reliably without the original designer babysitting it.
- Assign clear ownership and define how often sources are refreshed.
- Establish a definitive escalation path for when the model encounters uncertainty.



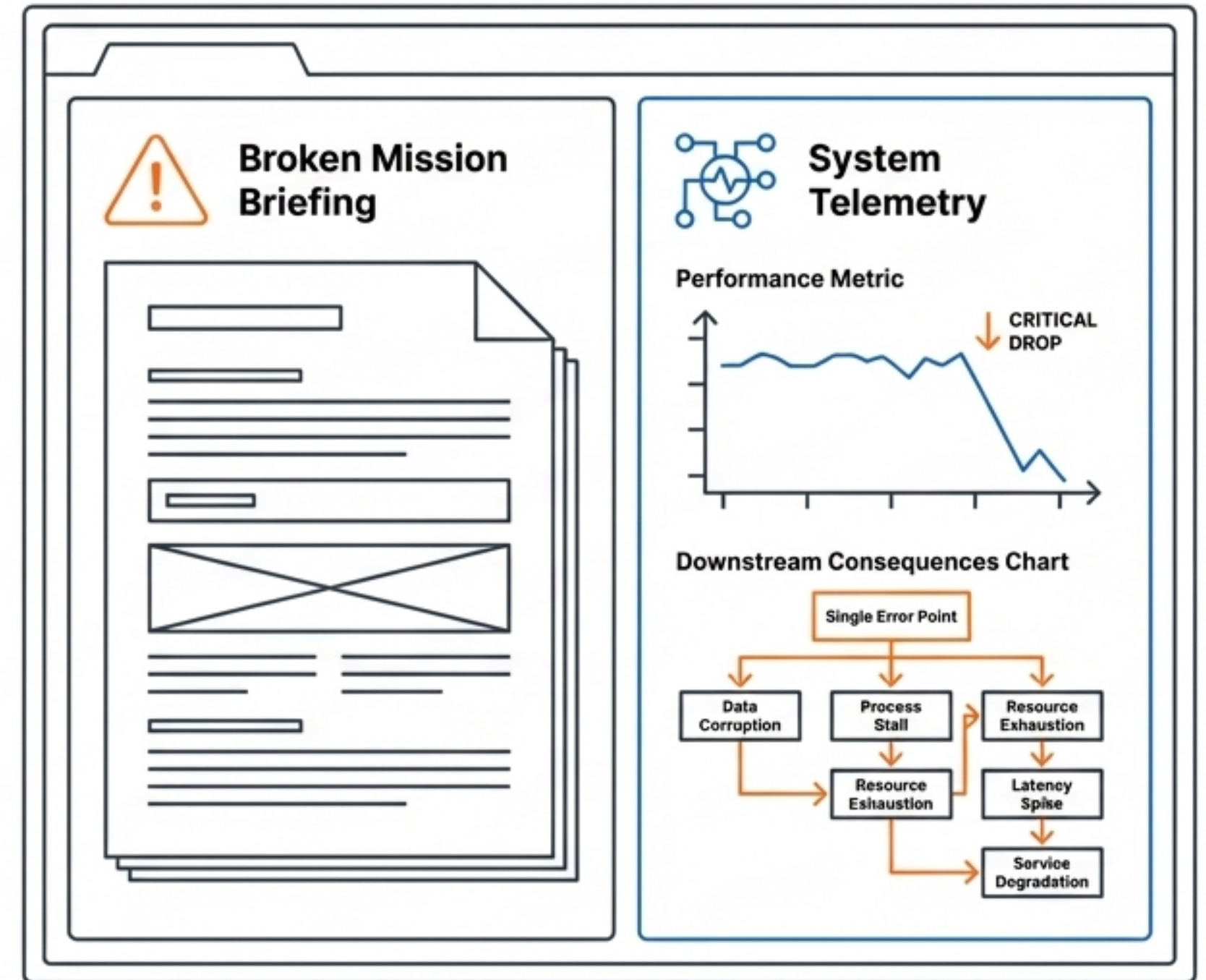
Diagnostic Troubleshooting Guide

Symptom	Likely Lifecycle Failure	Fix
Generic or Vague Answer	Weak Selection	Add minimum authoritative excerpts needed to ground the answer.
Verbose but Unhelpful	Selection Bloat	Prune context that does not change the answer or required format.
Confident Factual Error	Authority / Freshness Issue	Replace weak sources with the source of record; state which wins.
Inconsistent Formatting	Vague Shaping	Explicitly define the Output Contract (e.g., bullets, table).

Practice Lab 1: Lifecycle Failure Clinic

Learn context engineering through realistic operational failures.

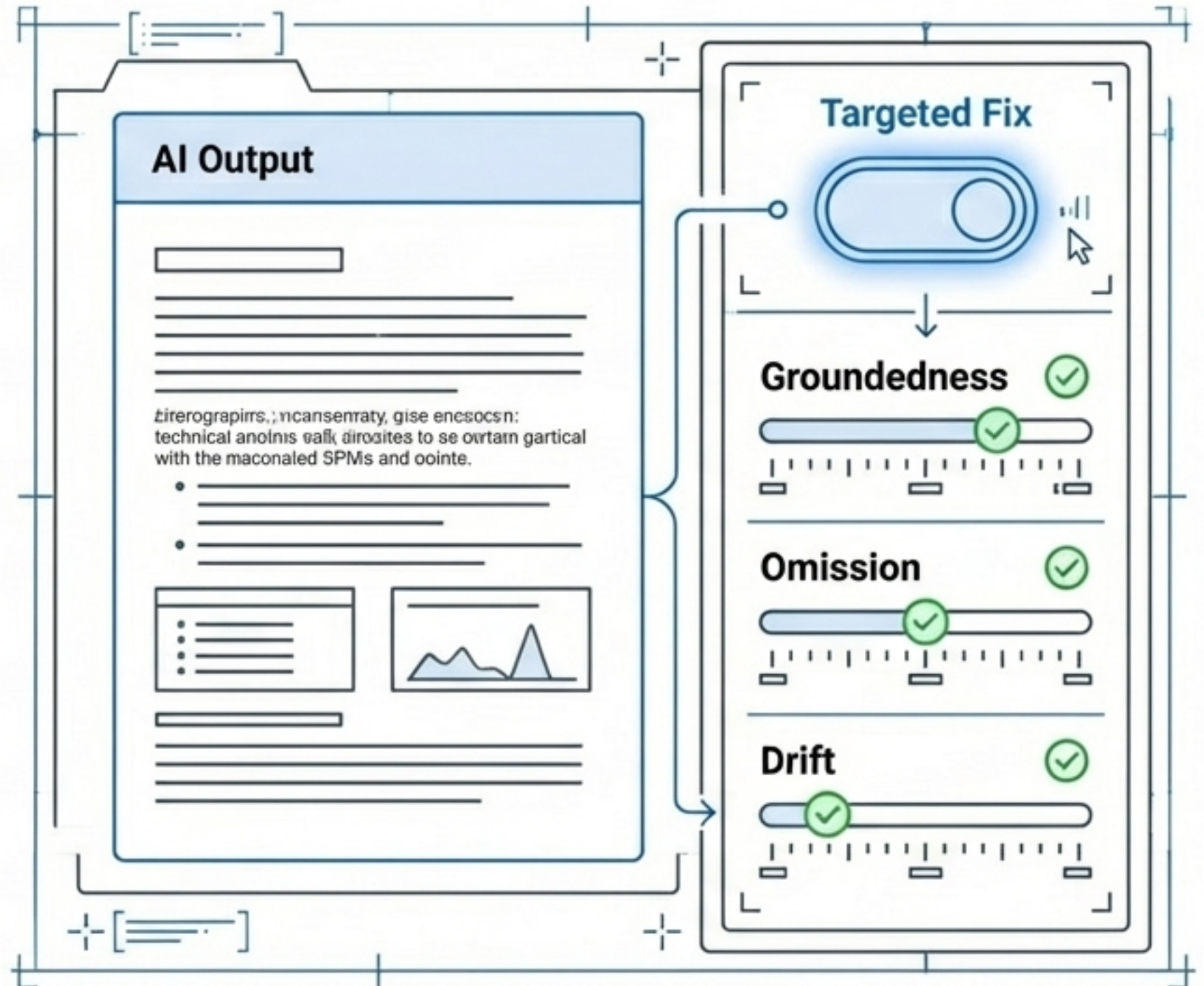
- **Diagnose:** Review broken missions and trace failures to root causes (not just surface polish).
- **Apply:** Choose one targeted repair that addresses the active phase weakness.
- **Observe:** Study live result changes, consequence chains, and system telemetry.



Practice Lab 2: Rubric Runner Lab

Judge AI outputs with discipline instead of instinct.

- **Set Standards:** Define success criteria before reviewing any answers.
- **Calibrate:** Score baseline outputs against visible evidence only (no inventing missing support).
- **Diagnose & Test:** Tag focused failures and test exactly one targeted fix at a time to ensure changes are comparable.



Context Quality Checklist

☐	Task Clarity: Is the job, audience, and success criteria explicitly defined?	☐	Precedence: Does the package dictate what wins in a conflict?
☐	Evidence Quality: Are sources authoritative, current, and necessary?	☐	Freshness: Have stale facts and old requirements been removed?
☐	Signal-to-Noise: Does every block materially help the answer?	☐	Safety: Is untrusted user input kept separate from trusted instructions?
☐	Structure: Are rules, evidence, facts, and task clearly separated?	☐	Operational Readiness: Does the workflow have an owner, refresh cadence, and escalation path?

Note: All 10 interactive evaluation applications are available online in the Chapter 2 & 3 Guides.